Bangladesh University of Business and Technology
Department of Computer Science and Engineering

CSE 498A: Capstone Project/Thesis

PROPOSAL

Semester: Summer 2026

Title: Automated Inventory Data Entry through Intelligent Character and Document Structure Recognition

Submission Date: 28th July, 2026

TEAM MEMBERS

Full ID	Full Name	Intake/Section
20234103222	Arifur Rahman	52 / 06
20234103240	Meer Mushfiqur Rahman	52 / 06
20234103212	Fatema Akter Akhi	52 / 06
20234103236	Sheikh Ramisha Zubaydah	52 / 06

SUPERVISOR & COURSE TEACHER APPROVAL

SUPERVISOR'S SIGNATURE WITH DATE



Md. Mijanur Rahman
Assistant Professor
Department of CSE, BUBT

COURSE TEACHER'S SIGNATURE WITH DATE

Md. Masudul Islam
Assistant Professor
Department of CSE, BUBT

1. Problem Statement

Small shops and fulfilment centres receive their purchase and sale records as unstructured documents, printed invoices, thermal-printer receipts, and handwritten chits, issued by many different suppliers with no common layout or standard. When one of these documents arrives at an inventory management system, a person currently has to read it and manually enter its contents, and this manual step is slow and tedious, so businesses frequently fall behind on keeping their records current or skip structured recording altogether. Existing inventory management systems do not solve this problem, since they assume the data entry has already happened by the time information reaches them and expect clean, structured input rather than a photograph of a paper slip. Off-the-shelf OCR alone is not sufficient either, since it produces raw text without structure, so a downstream system still cannot tell which text is a line-item description, which is a quantity, or what document type it is looking at in the first place, whether a purchase order, a delivery note, or a customer copy.

The engineering problem this project addresses is recognizing the characters and document structure of heterogeneous supplier documents so that their content and document type can be identified automatically. The system's extraction component is engineered to be reliable rather than trusted at face value: extracted numeric fields are cross-checked against the document's own arithmetic, and low-confidence results are flagged rather than silently finalized. The output of this pipeline is a standardized JSON representation of the document's content, served through an API that an authorized ERP or inventory management system can call to eliminate the manual data-entry step. Reconciling item names across suppliers, for example “Coca-Cola” versus “Coke”, is left to the consuming system.

2. Objectives

a. To build a document recognition pipeline that combines character recognition (OCR) with document structure/type recognition, so that the system can both read the text of a heterogeneous set of supplier documents (printed and handwritten, English-language) and correctly identify what kind of document it is looking at (e.g., purchase order, customer copy, delivery note, receipt).

b. To design a schema-constrained extraction component in which a cloud-hosted language model converts recognized document content into structured line-item records conforming to a fixed JSON schema.

c. To engineer a verification step around the language model's extraction output, cross-checking extracted numeric fields against the document's own arithmetic and flagging low-confidence records for review, so that no unverified value is silently finalized.

d. To design the digitization service as a stateless, idempotent API: each submission (e.g., a mobile-app-scanned document) produces a JSON output without the system itself owning or persisting inventory state, while ensuring a retried or duplicate submission cannot produce duplicate or conflicting output to the downstream ERP/inventory system.

e. To evaluate the system on character/structure recognition accuracy, document-type classification accuracy, and extraction accuracy, including the rate at which arithmetic self-validation catches extraction errors.

3. Methodology

The project follows an iterative, evaluation-driven development lifecycle — an Agile-style adaptation of the classic SDLC in which the recognition pipeline is built and refined across short, measurable cycles rather than in a single waterfall pass. Each phase produces a concrete deliverable, and results from the evaluation phase feed back to refine the API contract and the extraction design. The phases are shown in Figure 1, including:

- **Requirement analysis & document collection.** we interview small-shop and business staff to document their current manual process, and collect a corpus of real supplier invoices, receipts, and delivery notes (English) across printed, thermal, and handwritten formats.
- **Data model & API contract design. we define the standardized JSON output schema, a document-type taxonomy (purchase order, customer copy, delivery note, receipt), and the data model an authorized ERP or inventory system will consume via the API.**
- **Recognition pipeline.** a cloud-hosted OCR service reads document text, followed by a document structure/type classifier that determines what kind of document is being processed, then a schema-constrained language model that emits structured line items. Extracted numeric fields are cross-checked against the document's own arithmetic, and records that fail this check or fall below a confidence threshold are flagged for review rather than finalized silently. All recognition, extraction, and verification processing runs server-side, and the mobile application's role is limited to capturing the document.
- **Digitization API core.** the service is designed as a stateless, idempotent API, so a retried or duplicate document submission cannot produce duplicate or conflicting JSON output to the downstream ERP/inventory system.
- **Evaluation & benchmarking. we measure character and document-structure recognition accuracy, extraction accuracy, and the rate at which arithmetic self-validation catches extraction errors, against the confidence threshold used for routing low-confidence extractions.**

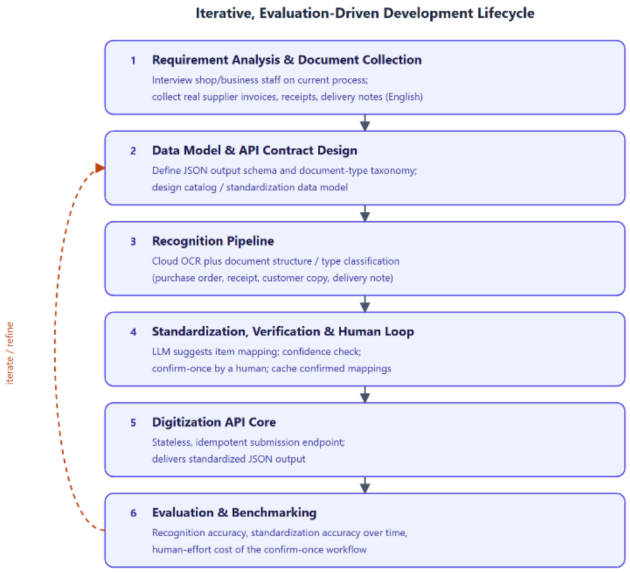


Figure 1. Iterative methodology workflow showing the five development phases and their feedback loop.

The resulting system architecture is shown in Figure 2. The mobile application is responsible only for capturing a document (photograph or scan); all recognition and extraction processing runs in the cloud. This deliberately avoids the reliability, performance, and maintenance problems of running a capable language model directly on a mobile device.

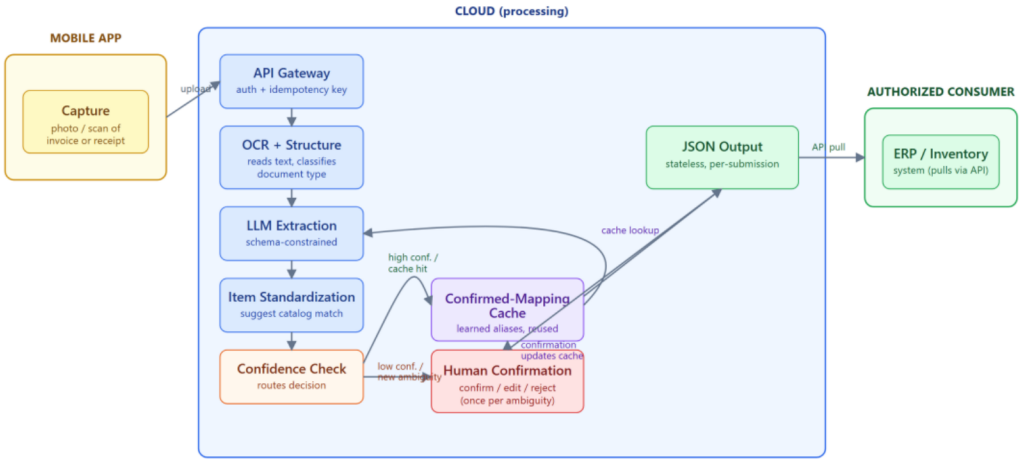
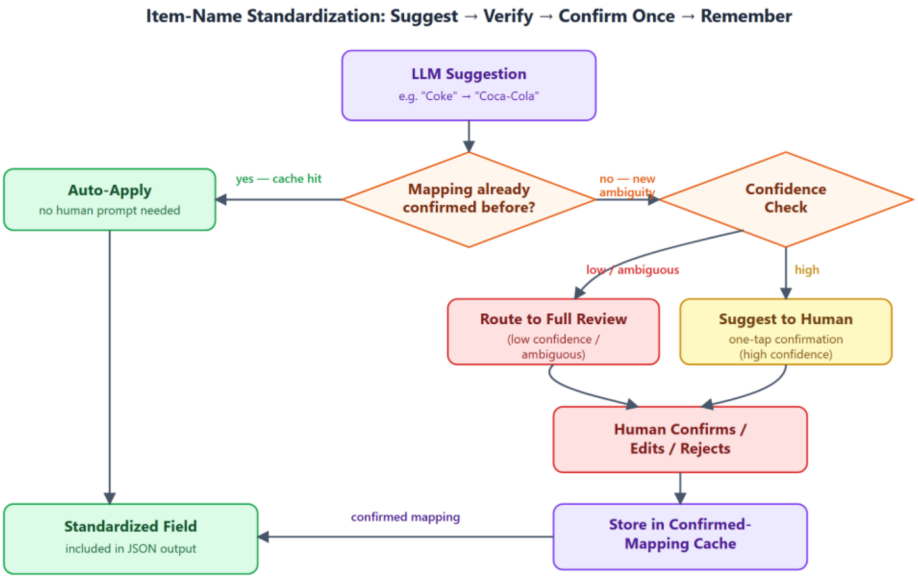


Figure 2. System architecture: the mobile app captures a document; cloud services perform OCR, document structure recognition, and schema-constrained extraction; the result is delivered as a stateless JSON output via an API to an authorized ERP/inventory system.



4. Expected Outcomes

i. A working mobile-to-cloud digitization gateway that ingests supplier invoices, receipts, and inventory documents and delivers a standardized JSON output via API.

ii. A character and document structure recognition pipeline whose recognition accuracy on printed and handwritten English-language documents is targeted at a competitive level against benchmarked commercial and academic baselines, with document-type classification as a first-class output.

iii. A stateless, idempotent API design demonstrably resilient to retried or duplicate document submissions.

iv. An evaluation of recognition and document-type classification accuracy.

5. Complex Engineering Problem Mapping

Mapping with Knowledge Profile (WK)

- **WK1 (Natural Sciences)**: Not directly applicable; the project centres on software engineering, NLP, and systems design rather than natural-science principles.
- **WK2 (Mathematics & Statistics): Applied in evaluation metrics (recognition and document-type classification accuracy) and confidence scoring for the extraction pipeline.**
- **WK3 (Engineering Fundamentals)**: Mobile-to-cloud system architecture, API design, and database design form the engineering foundation of the pipeline.
- **WK4 (Specialist Knowledge)**: NLP, language models, OCR, and document structure recognition — specialist knowledge at the forefront of the discipline.

- **WK5 (Engineering Design): The API contract, the document-type taxonomy, and the extraction pipeline require deliberate engineering-design knowledge.**
- **WK6 (Engineering Practice)**: Cloud deployment practices, version control, and systematic testing apply current engineering practice.
- **WK7 (Comprehension)**: Data handling for business records, informed consent in data collection, and professional responsibility for output correctness are central.
- **WK8 (Research Literature): The project engages directly with the literature on document understanding, structured LLM output, and hallucination mitigation.**

Mapping with Complex Engineering Problems (WP)

- **WP1 (Depth of knowledge required): Requires in-depth ML/NLP knowledge (OCR, document structure recognition) plus systems and API design, with sustained engagement with the research literature.**
- **WP2 (Range of conflicting requirements): Recognition automation vs. extraction correctness; usability for non-technical staff vs. system sophistication.**
- **WP3 (Depth of analysis required): No off-the-shelf platform combines document-structure recognition with schema-constrained extraction and arithmetic-based verification tuned to heterogeneous small-business supplier documents.**
- **WP4 (Familiarity of issues): Reliable document-type recognition for small-business inventory documents is not a well-established, off-the-shelf combination in industry or research.**
- **WP5 (Extent of applicable codes)**: Handling supplier/business documents engages data-handling and access-control expectations for the API and for any interim storage of submitted documents.
- **WP6 (Extent of stakeholder involvement & conflicting requirements): Shop/business staff submitting documents and the downstream ERP/inventory systems consuming the API form a stakeholder set with differing needs.**
- **WP7 (Interdependence): Character recognition and document structure recognition are tightly coupled — a structure-recognition error can misroute how downstream fields are extracted.**

Mapping with Complex Engineering Activities (EA)

- **EA1 (Range of resources): Cloud OCR/document-AI services, language models, mobile app development tooling, and API infrastructure.**
- **EA2 (Level of interactions): Resolving interactions between document-structure recognition accuracy and the design of the extraction pipeline.**

- **EA3 (Innovation): Creative composition of existing recognition, extraction, and hallucination-mitigation techniques into a document-capture-to-API pipeline for heterogeneous supplier documents.**
- **EA4 (Consequences to society and the environment)**: Lowers the data-entry burden for small and medium businesses, allowing existing inventory/ERP tooling to be automated without requiring a full software replacement.
- **EA5 (Familiarity of issues)**: Extends beyond prior coursework by applying principles-based approaches — system design under model uncertainty, API contract design, and evaluation methodology — to an unfamiliar, open-ended problem.

References

1. M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI Conference on Artificial Intelligence, vol. 37, pp. 13094–13102, 2023.
2. G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, "OCR-free document understanding transformer," in Proc. European Conference on Computer Vision (ECCV), 2022. (arXiv:2111.15664)
3. Y. Xu et al., "LayoutLM: Pre-training of text and layout for document image understanding," in Proc. KDD, 2020.
4. B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv preprint arXiv:2307.09702, 2023.
5. S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in Proc. EMNLP, pp. 10932–10952, 2023.
6. S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "JSONSchemaBench: A rigorous benchmark of structured outputs for language models," arXiv preprint arXiv:2501.10868, 2025.
7. Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H. yi Lee, and Y.-N. Chen, "Let me speak freely? A study on the impact of format restrictions on performance of large language models," in Proc. EMNLP: Industry Track, pp. 1218–1236, 2024.
8. P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Findings of ACL: EMNLP 2023, pp. 9004–9017, 2023.
9. S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," Nature, vol. 630, pp. 625–630, 2024.
10. Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating hallucination in large language models via self-reflection," in Findings of ACL: EMNLP 2023, pp. 1827–1843, 2023.
11. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. ICML, PMLR 70, 2017.
12. S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, et al., "Language models (mostly) know what they know," arXiv preprint arXiv:2207.05221, 2022.
13. P. Helland, "Idempotence is not a medical condition," Communications of the ACM, vol. 55, no. 5, pp. 56–65, 2012.